Тестове завдання для Frontend Engineer

Опис завдання

Необхідно створити **веб-застосунок для управління сільськогосподарськими полями** з можливістю відображення їх на карті, додавання моніторингових точок та базового керування.

Функціональні вимоги

1. Карта з полями

- Відображення інтерактивної карти (Leaflet)
- Візуалізація 3-5 попередньо заданих полів у форматі GeoJSON
- Можливість перемикання між полями (список або клік на карті)
- Виділення активного поля іншим кольором
- Відображення базової інформації про поле (назва, площа)

2. Моніторингові точки

- Додавання точок на карті в межах вибраного поля (клік на карті)
- Кожна точка має містити:
    - Координати (latitude, longitude) WGS 84 (без висоти)
        - Додаткове виведення координат у MGRS системі. Обчислення на фронтенді.
    - Тип точки (випадаючий список: "Проба ґрунту", "Шкідники", "Хвороби рослин", "Інше")
    - Опис (текстове поле, опціонально)
    - Дата створення (автоматично)
- Відображення точок на карті з різними іконками залежно від типу
- Видалення точок

3. Панель управління

- Список усіх доданих точок з можливістю:
    - Фільтрації за типом
    - Пошуку за описом
    - Сортування за датою

4. UI/UX вимоги

- Адаптивний дизайн (desktop + tablet)
- Зручна навігація між полями
- Чітка візуальна ієрархія

Технічні вимоги

Обов'язкові технології:

- **React 18+** з **TypeScript**
- **Leaflet** для карт
- **State management**: Zustand або Redux Toolkit або Context API
- **Vite** як build tool
- **Tailwind CSS** для стилізації

Приклад GeoJSON для тестових полів:

```
{
  "type": "Feature",
  "properties": {
    "id": "field-1",
    "name": "Поле №1 - Пшениця",
    "area": 45.2,
    "crop": "Пшениця"
  },
  "geometry": {
    "type": "Polygon",
    "coordinates": [[
      [30.5234, 50.4501],
      [30.5334, 50.4501],
      [30.5334, 50.4601],
      [30.5234, 50.4601],
      [30.5234, 50.4501]
    ]]
  }
}
```

Критерії оцінювання

1. Код та функціонал (60%)

- Працююча карта з полями та точками
- Якісний TypeScript (мінімум ***any***)
- Чистий, читабельний код
- State management
- Фільтрація та UI/UX
- Обробка помилок

2. Презентація та розуміння (40%)

- Здатність пояснити свої архітектурні рішення
- Розуміння кожної частини коду
- Вміння обґрунтувати технічні вибори
- Комунікаційні навички
- Критичне мислення щодо власного рішення

Що потрібно здати

1. **Архів репозиторій** робочого проекту.
    - Код застосунку
    - README з інструкцією запуску
2. **Короткий опис** (у README):
    - Використані технології
    - Прийняті архітектурні рішення
    - Що б ви додали, якби мали більше часу

Презентація роботи

Після здачі завдання буде проведено **30-хвилинну зустріч**, де потрібно буде:

Структура презентації:

1. **Демонстрація функціоналу** (5-7 хв)
2. **Огляд коду** (10-15 хв)
3. **Обґрунтування рішень** (10-15 хв)
4. **Питання та відповіді** (5 хв)

Що оцінюватимемо:

- Розуміння написаного коду
- Вміння пояснити технічні рішення
- Критичне мислення щодо власного коду
- Комунікаційні навички

Питання та уточнення

Якщо виникли питання щодо завдання, можете:

- Зробити обґрунтоване припущення та описати його в README
- Звернутися до контактної особи (вказати email/telegram)

Важливі примітки

- Використовуйте mock дані (не потрібен backend)
- Зосередьтеся на якості коду та UX, а не на кількості функцій
- Покажіть свій підхід до вирішення проблем
- Код має бути готовий до code review

Використання AI інструментів

Ви можете використовувати інструменти штучного інтелекту (ChatGPT, Claude, Copilot, тощо) для допомоги у виконанні завдання.

Проте важливо розуміти:

- AI - це інструмент, як Stack Overflow чи документація

- Весь код, який ви здаєте, має бути **повністю переглянутий та перевірений вами**
- Ви здаєте код **від свого імені** і несете за нього повну відповідальність
- На презентації вас попросять **пояснити будь-яку частину коду** - ви маєте розуміти кожен рядок
- Якщо ви не можете пояснити, чому код написаний саме так - це негативний сигнал

Що ми перевіряємо:

- Чи розумієте ви архітектурні рішення у вашому коді?
- Чи можете пояснити, чому обрали саме такий підхід?
- Чи здатні обґрунтувати альтернативні варіанти?
- Чи можете швидко орієнтуватися у власному коді?

Рекомендація: Навіть якщо використовуєте AI, завжди:

1. Розберіться, як працює згенерований код
2. Адаптуйте його під свій стиль та вимоги
3. Переконайтеся, що можете пояснити кожне рішення